

Weekly Progress Report

Project: Custom 2D Platformer - Group 3

Date / Week: 27/7 to 1/8 | Week 08

1. Weekly Overview

- **Primary Goal:**
 - Integrate the newly developed **PhysicsEngine** (gravity and AABB collision) with actual game entities (Hero, Enemies) instead of testing with dummy bounding boxes.
 - Finalize Khải's in-process refactoring of the **Character** and **Hero** classes, fully implementing Mario's state-swapping logic via the Strategy Pattern.
 - Implement complex entity interaction logic (e.g., Mario squishing Goombas, colliding with sliding Koopa shells, or taking damage).
 - Merge all individual modules (Engine, Level Management, Hero, Enemies) into the main development branch to successfully assemble and run the first playable gameplay demo (All Members).
 - Develop the tilemap renderer to load and display static Mario levels (bricks, pipes) from text/csv files (delayed task).
 - Implement the dynamic HUD (score, coins, time) utilizing the Observer Pattern.
 - Integrate the completed **Goomba** and **Koopa** classes into the main game loop to test their Patrol AI and state management.
 - Spawn classic items (Mushroom, Coin) utilizing the Factory Pattern.
- **Completion Rate:** 40% of the weekly plan completed.

2. Individual Task Breakdown

25125057 - Trần Minh Khoa is responsible for the **Core Engine & Physics**.

- **Note:** Remain tasks completed, waiting for next phase

25125045 - Trần Như Khải is in charge of the **Hero & Items Logic**.

- **Completed Tasks:**
 - Reimplemented Hero class with State and Strategy Factory, separated State and Form interaction logic.
 - Applied Sprite sheet for Mario and Luigi, Tested simple moving logic
- **Proof:** <https://github.com/trannhukhai98438/CS202-Group03-2DGame/tree/feature/Hero>

25125024 - Đỗ Viết Hoàng Long is handling the **Enemies & AI Design**.

- **Completed Tasks:**
 - Implemented the **Enemy** base class with patrol AI, sprite animation, and collision handling.
 - Implemented **Goomba** enemy with 4-frame walk animation, patrol behavior, and squish-on-stomp mechanic.

- Implemented **Koopa** enemy with walk animation, Shell and SpinningShell states, and kick-on-side-collision mechanic.
- **Proof:**
 - <https://github.com/trannhukhai98438/CS202-Group03-2DGame/commit/173e03b>
 - <https://github.com/trannhukhai98438/CS202-Group03-2DGame/commit/3f66765>

25125056 - Trần Đăng Khoa is assigned to the development of the **Tilemap, HUD & Audio**.

- **Completed Tasks:**
 - Implemented HUDManager class.
 - Integrate HUDManager into PlayingState.
- **Proof:**
 - <https://github.com/trannhukhai98438/CS202-Group03-2DGame/commit/47a2fe6432e2fb0192b544c57c608178cea11e00>
 - <https://github.com/trannhukhai98438/CS202-Group03-2DGame/commit/c9dd808b8cd6058e52755473695e7af4aa2db950>

3. Challenges & Solutions

Issue 1: The Hero character possesses multiple physical forms (e.g., Small, Giant, Fire), and each form shares similar action states (e.g., idle, running, jumping). Relying on a single Strategy pattern solely for the character's action states made coordinating the current form with the active state highly complex. This approach complicated the logic for determining the correct animation frame, resulting in lengthy, tightly coupled, and hard-to-understand code. **Solution 1:** We implemented a dual State/Strategy pattern approach to manage both the character's "Form" (Macro-state) and "Action" (Micro-state). By decoupling the physical Form from the Action State, both systems can now operate independently while coordinating seamlessly to resolve the correct physics parameters and animation frames without excessive if-else branching.

4. AI Usage Notes

- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 23:00 on July 24, 2026, prompt: "How do I implement HUDManager and integrate it into PlayingState in C++ SFML, and does decoupling player stats updates from the render pass optimize performance?" to finalize the in-game UI architecture, AI suggested the implementation blueprints for HUDManager and event-driven integration with PlayingState, Student reviewed the code and integrated the HUD system into the engine.
- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 19:00 on July 29, 2026, prompt: "How to implement spritesheet animation and load custom texture shapes for Koopa and Goomba enemies in a C++ SFML platformer?" to research sprite sheet animation rendering and texture scaling, AI provided guidance on calculating source rectangles and dynamically adjusting sprite origins, Student reviewed the suggestions and implemented the sprite logic for Goomba and Koopa.
- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 10:11 on July 30, 2026, prompt: "Regarding the Hero design approach, should input handling logic within update(deltaTime) be placed in the Hero class, the Action State class, or the Physical Form class?" to determine the optimal architecture for building the Hero using State and Strategy patterns, AI proposed several architectural options along with their pros and cons, Student reviewed the suggestions and selected the final implementation approach.

5. Next Week's Action Plan

- **Core Mechanics:**
 - Merge all individual feature branches into the primary **dev** branch to consolidate the codebase[cite: 6].
 - Conduct system-wide adjustments, code reviews, and bug squashing to ensure stability across the integrated architecture[cite: 6].
- **Gameplay / Graphics:**
 - Finalize the core implementations for the Hero class, interactive Bricks, classic Items, and Enemy entities (Goombas, Koopas)[cite: 6].
 - Construct a functional demo map layout to actively playtest all integrated features, verify complex entity interactions, and identify runtime bugs[cite: 6].
- **Expected Deadline:** 8/8/2026